Plataforma de estructuras de datos

Documentación del proyecto final

Estructuras de Datos II

Universidad Autónoma de Occidente

Integrante: Carlos Andrés Aristizábal Hernández

Código: 2230750

Docente: Jonathan López

Sitio publicado: https://proyecto-final-uao.netlify.app

Repositorio: https://github.com/Carlosraw/proyecto-final-estructuras-de-datos-2

1. Alcance

El proyecto es una plataforma web con cuatro módulos visuales. Cada módulo se apoya en una estructura de datos implementada desde cero en JavaScript, sin usar librerías de estructuras ya hechas. Todos los datos se guardan en una base de datos, de modo que la información se conserva al recargar la página y se comparte entre visitantes.

Módulo	Ruta	Estructura	Caso de uso
Pila	/pila	Pila (Stack)	Libros apilados: solo se retira el de arriba.
Cola	/cola	Cola (Queue)	Personas esperando en un cajero: se atiende por orden de llegada.
Buscador	/buscador	Trie y MinHeap	Búsqueda de productos por prefijo, mostrando los más populares primero.
Grafo	/grafo	Grafo no dirigido	Personas, ciudades y amistades.

Con estos módulos se cubren los requisitos de la rúbrica: dos estructuras de listas, pilas o colas (pila y cola), dos de árboles, tries o heaps (trie y heap) y un grafo.

El trabajo es individual, por lo que se priorizó una plataforma sencilla, completa y publicada.

2. Tecnologías

Tecnología	Uso
React 18 y Vite	Interfaz de usuario y entorno de desarrollo y construcción.
react-router-dom	Navegación entre los módulos sin recargar la página.
react-d3-graph	Dibujo interactivo del grafo.
Supabase (PostgreSQL)	Base de datos y API, consumidas con @supabase/supabase-js.
Netlify	Publicación del sitio en un servidor público.
Git y GitHub	Control de versiones y repositorio.

3. Estructuras de datos y justificación

Pila

Los libros se apilan y solo se puede retirar el que está arriba, que es el comportamiento de una pila (último en entrar, primero en salir). Operaciones: push, pop, peek, isEmpty y size. Todas se resuelven en tiempo constante.

Cola

Las personas del cajero deben atenderse en el mismo orden en que llegaron. La cola (primero en entrar, primero en salir) garantiza ese orden. Operaciones: enqueue, dequeue, peek, isEmpty y size.

Trie

Para sugerir productos a partir de un prefijo no es necesario recorrer todo el catálogo. El trie guarda los nombres letra por letra: se baja hasta el nodo del prefijo y desde ahí solo se visitan los productos que lo comparten. El costo depende de la longitud del prefijo, no de la cantidad total de productos.

MinHeap

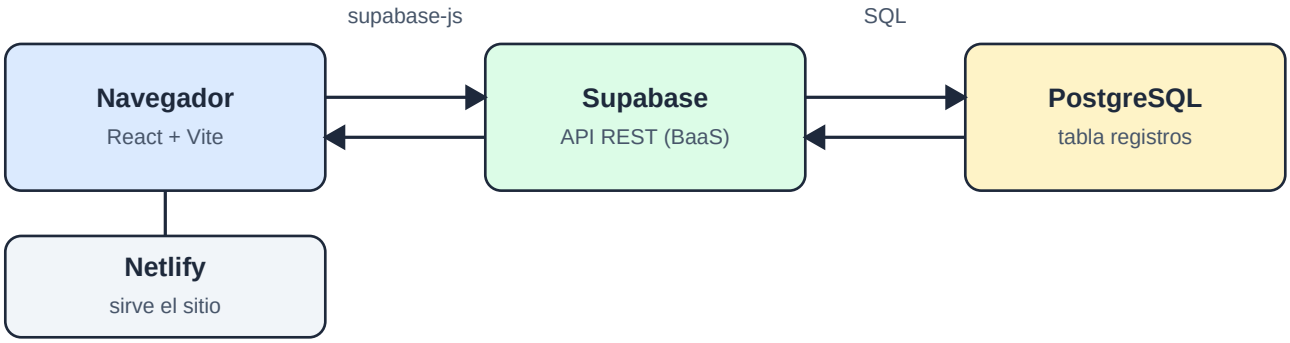
Para mostrar los K productos más populares de un prefijo se usa un heap mínimo de tamaño K. Cada producto encontrado entra al heap y, si el heap supera K elementos, sale el menos popular. Así se obtiene el top K sin ordenar todos los resultados.

Grafo

Las personas y las ciudades son nodos; la residencia y la amistad son aristas. Se implementó con lista de adyacencia, que es eficiente cuando cada nodo tiene pocas conexiones y permite consultar rápido las personas que viven en una ciudad o los amigos de una persona.

4. Arquitectura

La aplicación es un frontend en React que se comunica directamente con Supabase. No se desarrolló un servidor propio: Supabase funciona como backend administrado (BaaS) y expone la base de datos mediante una API que el frontend consume con la URL del proyecto y una clave publicable. Por eso no hay un servicio de backend adicional que publicar.



Netlify entrega los archivos estáticos del sitio al navegador. Una vez cargado, el navegador habla directamente con Supabase.

5. Persistencia y transacciones

Los cuatro módulos guardan sus datos en una única tabla llamada **registros**:

Columna	Tipo	Descripción
id	bigint (identidad)	Identificador autoincremental. Conserva el orden de inserción.
estructura	text	Módulo al que pertenece: pila, cola, producto o grafo.
datos	jsonb	Contenido del registro (por ejemplo, el título de un libro).
creado_en	timestamptz	Fecha y hora de creación.

Operaciones que la plataforma envía a la base de datos:

- **Insertar:** apilar, encolar, guardar un producto, agregar una ciudad, una persona o una amistad.
- **Consultar:** al abrir cada página se leen los registros de su módulo, ordenados por id.
- **Eliminar:** desapilar, atender a una persona y eliminar un producto.

Al cargar una página, la estructura se reconstruye en memoria a partir de los registros, aplicándolos en el mismo orden en que fueron guardados. Así, la pila, la cola, el trie y el grafo recuperan exactamente el estado en que quedaron.

En el módulo del grafo, la fila guarda un campo **tipo** dentro de datos (ciudad, persona o amistad), lo que permite almacenar tres clases de elementos en la misma tabla.

6. Despliegue

El sitio está publicado en Netlify y se reconstruye automáticamente cada vez que se actualiza la rama principal del repositorio.

Elemento	Detalle
Sitio público	https://proyecto-final-uao.netlify.app
Comando de construcción	npm run build
Carpeta publicada	dist
Variables de entorno	VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY, configuradas en Netlify. No se guardan en el repositorio.
Rutas	Un archivo netlify.toml redirige todas las rutas a index.html para que React Router funcione al recargar.
Base de datos	Proyecto en Supabase (https://tvgylkaychoetemcfhju.supabase.co).

7. Control de versiones

El proyecto vive en su propio repositorio, separado del repositorio de los retos del curso:

- **Proyecto final:** https://github.com/Carlosraw/proyecto-final-estructuras-de-datos-2
- **Retos del curso:** https://github.com/Carlosraw/estructuras-de-datos-2

El trabajo se desarrolló en la rama **feature/proyecto-final**, con un commit por funcionalidad: base del proyecto, conexión a Supabase, pila, cola, buscador y grafo. Esa rama se fusionó en **main**, que es la rama desplegada.

8. Cómo ejecutarlo en local

Se necesita Node.js. Desde la carpeta del repositorio:

```
npm install
npm run dev
```

Además, hay que crear un archivo **.env** en la raíz del proyecto con las credenciales públicas de Supabase:

```
VITE_SUPABASE_URL=https://su-proyecto.supabase.co
VITE_SUPABASE_ANON_KEY=su-clave-publicable
```

El archivo **.npmrc** incluye **legacy-peer-deps=true**, necesario porque react-d3-graph declara compatibilidad con una versión anterior de React.

9. Limitaciones

- La tabla tiene una política de acceso abierta y no hay inicio de sesión: cualquier visitante puede leer y modificar los datos. En un uso real habría que agregar autenticación y reglas por usuario.
- Los datos son compartidos entre todos los visitantes del sitio.
- El grafo permite agregar ciudades, personas y amistades, pero no eliminarlas.
- Los nombres de ciudades y personas del grafo son únicos.
- Al no haber servidor propio, la lógica de las estructuras se ejecuta en el navegador y la base de datos solo almacena los registros.